



University
of Exeter

COMM113

Deep Learning – Week 2

Neural Networks and Backpropagation

Gaojie (Jay) Jin
g.jin@exeter.ac.uk

Q&A

1. Why did we discuss the derivative of the activation functions?

The derivative is important because of backpropagation: during training we need to compute how much each weight contributes to the error. Without derivatives, we cannot update the parameters effectively.

2. What is the purpose of the bias when the weights appear to do something similar?

Weights control the slope (how strongly inputs affect the output).
Bias controls the offset (where the decision boundary is placed).

3. More big picture, what is happening as a whole? All these steps are happening but what is the goal?

The big picture: the network is trying to map inputs to outputs by learning weights and biases that minimise the error.

The goal: approximate an unknown function (classification, regression, etc.) so the predictions are close to the real answers.

Q&A

4. How do we decide the values for the weights and bias?

Initially: set randomly (or with specific initialisation schemes like Xavier/He).

During training: use gradient descent and backpropagation to update them iteratively, based on the training data and loss function.

5. In the PyTorch example we input a string of numbers – what is the output probability telling us in this case?

The output probability shows the model's confidence in each possible class.

For classification tasks, the highest probability corresponds to the predicted class.

Example: if probabilities are $[0.1, 0.7, 0.2]$, the model thinks class 2 is most likely with 70% confidence.

6. Activation function

The activation function introduces non-linearity into the model, allowing it to learn complex patterns.

Without it, a neural network would just be a stack of linear transformations, effectively behaving like a single linear model.

Loss: How good is your neural network?

Inputs

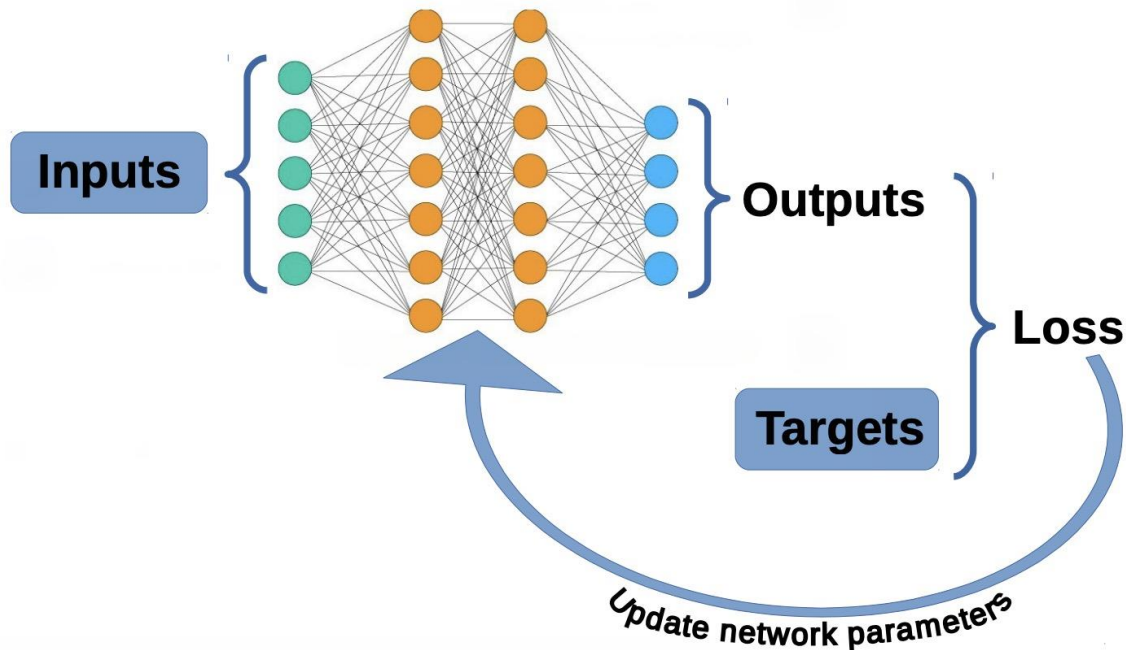
e.g.

- Image (pixel values)
- Sentence (encoded)
- State (for RL)

Targets

e.g.

- 'Cat'/'Dog' (encoded)
- Next word (encoded)
- TD(0) (for RL)



Why do we need a loss function at all, instead of just looking at accuracy?

Loss: How good is your neural network?

Inputs

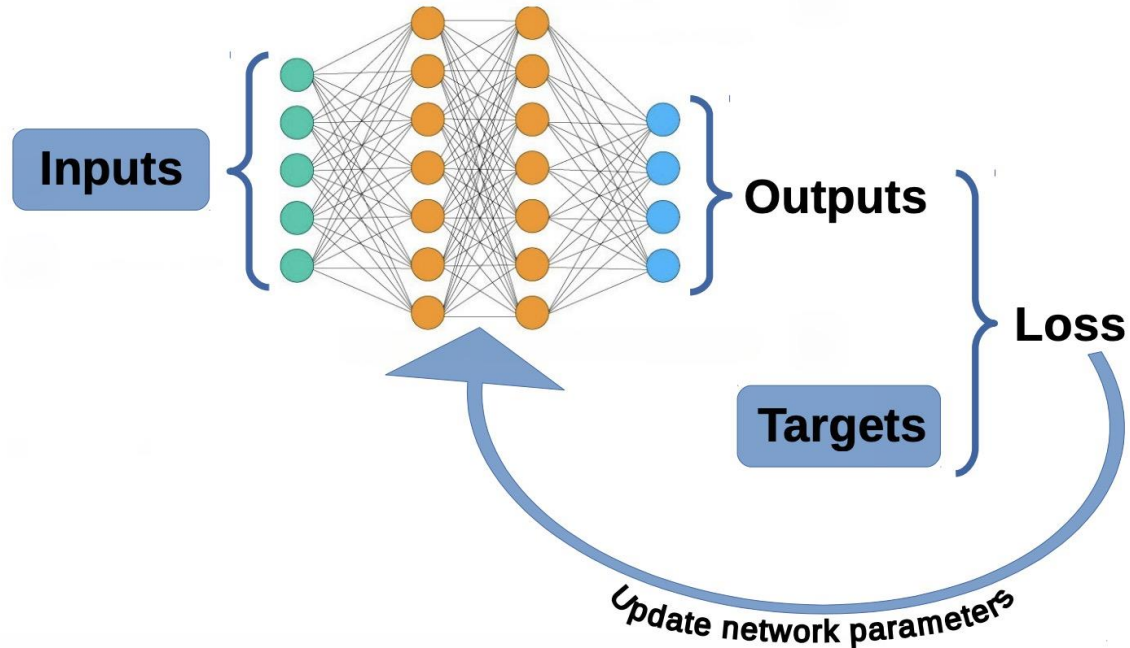
e.g.

- Image (pixel values)
- Sentence (encoded)
- State (for RL)

Targets

e.g.

- 'Cat'/'Dog' (encoded)
- Next word (encoded)
- TD(0) (for RL)



Why do we need a loss function at all, instead of just looking at accuracy?

Loss gives a finer measure of prediction confidence, not just correct/incorrect. It guides gradient-based optimisation.

Classification Loss

Binary Cross Entropy (BCE)	Loss function for binary classification tasks.	$\mathcal{L}_{BCE} = \frac{1}{N} \sum_{i=1}^N y_i \cdot \log(p(x_i)) + (1 - y_i) \cdot \log(1 - p(x_i))$
Hinge Loss	Penalizes wrong and right (but less confident) predictions. Commonly used in SVMs.	$\mathcal{L}_{\text{Hinge}} = \max(0, 1 - (f(x) \cdot y))$
Cross Entropy Loss	Extension of BCE loss to multi-class classification.	$\mathcal{L}_{CE} = -\frac{1}{N} \sum_{i=1}^N \sum_{j=1}^M y_{ij} \cdot \log(f(x_{ij}))$ N : samples; M : classes
KL Divergence	Minimizes the divergence between predicted and true probability distribution	$\mathcal{L}_{KL} = \sum_{i=1}^N y_i \cdot \log\left(\frac{y_i}{f(x_i)}\right)$

Binary

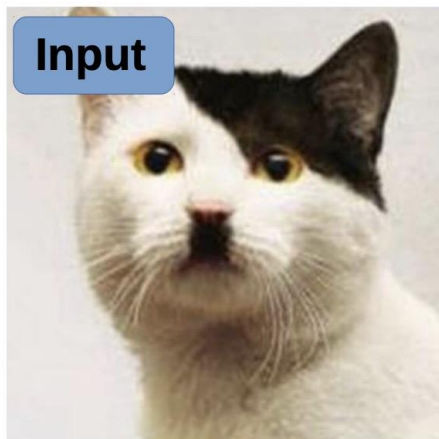
Multi-class

Cross-entropy Loss

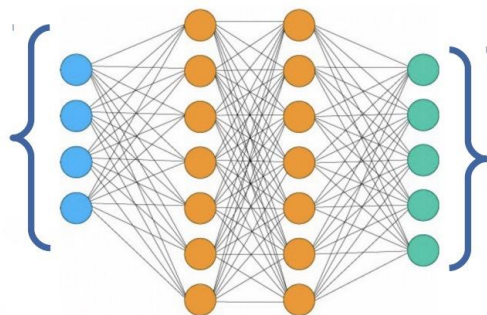
Categorical cross-entropy

Torch.nn.NLLLoss

Torch.nn.CrossEntropyLoss



Input



- Cat
- Dog
- Bear
- Hitler
- Putin
(.8, 0, 0, .2, 0)

This one does the softmax for you

$$= \log \left(\frac{\exp(x_i)}{\sum_j \exp(x_j)} \right)$$

Must be a probability distribution

$$Loss = - \sum_{i=1}^N y_i \cdot \log(p(y_i))$$

$$= - [1 * \log(0.8)]$$
$$= 0.22$$

Target

(1, 0, 0, 0, 0)

Cat -
Dog -
Bear -
Hitler -
Putin -

This loss function can also be used when you have a single binary (e.g. yes/no) output

How to compute the gradient of $p(y_i)$ with respect to cross-entropy loss?

Cross-entropy Loss

$$L = - \sum_{i=1}^N y_i \log(p_i)$$

$$\frac{\partial L}{\partial p_i} = -\frac{y_i}{p_i}, \quad i = 1, \dots, N.$$

Interpretation

- If class i is the true label ($y_i = 1$):

$$\frac{\partial L}{\partial p_i} = -\frac{1}{p_i}.$$

- If class i is not the true label ($y_i = 0$):

$$\frac{\partial L}{\partial p_i} = 0.$$

We have model weights, training data and label, loss function, how to update model weights to minimise the training loss?

We have model weights, training data and label, loss function, how to update model weights to minimise the training loss?

Gradient!

Computing Gradients

Output Weights: $\frac{\partial l(\mathbf{f}(\mathbf{x}), y)}{\partial W_{ij}^o}$

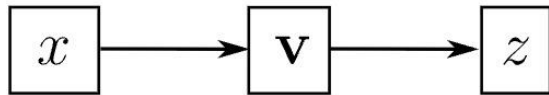
Output bias: $\frac{\partial l(\mathbf{f}(\mathbf{x}), y)}{\partial b_i^o}$

Hidden Weights: $\frac{\partial l(\mathbf{f}(\mathbf{x}), y)}{\partial W_{ij}^h}$

Hidden bias: $\frac{\partial l(\mathbf{f}(\mathbf{x}), y)}{\partial b_i^h}$

- The network is a composition of differentiable modules
- We can apply the "chain rule"

Chain Rule



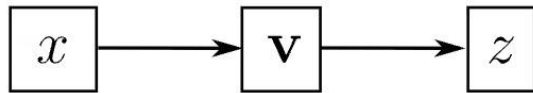
$$z = u(\mathbf{v}(x))$$

$$\frac{\partial z}{\partial x} = ?$$

$$v_j$$

$\mathbf{v}$

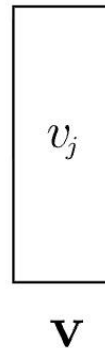
Chain Rule



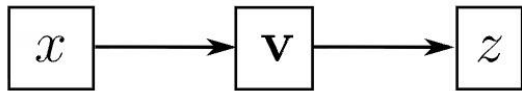
$$z = u(\mathbf{v}(x))$$

chain-rule

$$\frac{\partial z}{\partial x} = \sum_j \frac{\partial z}{\partial v_j} \frac{\partial v_j}{\partial x}$$



Chain Rule



$$z = u(\mathbf{v}(x))$$

chain-rule

$$\frac{\partial z}{\partial x} = \sum_j \frac{\partial z}{\partial v_j} \frac{\partial v_j}{\partial x} = \nabla u \cdot \frac{\partial \mathbf{v}}{\partial x}$$

$$v_j$$

$\mathbf{v}$

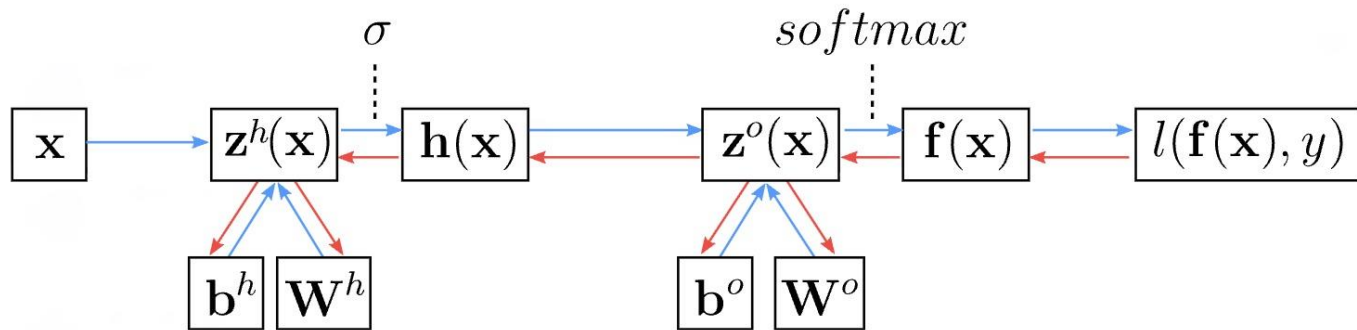
$$\frac{\partial v_j}{\partial x}$$

$$\frac{\partial \mathbf{v}}{\partial x}$$

$$\frac{\partial z}{\partial v_j}$$

∇u

Backpropagation

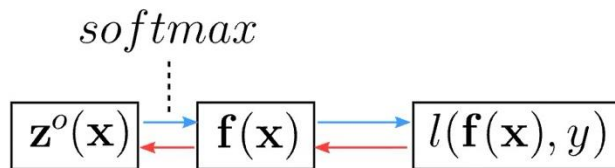


Compute partial derivatives of the loss

- $\frac{\partial l(\mathbf{f}(\mathbf{x}), y)}{\partial \mathbf{f}(\mathbf{x})_i} = \frac{\partial -\log \mathbf{f}(\mathbf{x})_y}{\partial \mathbf{f}(\mathbf{x})_i} = \frac{-1_{y=i}}{\mathbf{f}(\mathbf{x})_y}$
- $\frac{\partial l}{\partial \mathbf{z}^o(\mathbf{x})_i} = ?$

$$\frac{\partial l}{\partial \mathbf{z}^o(\mathbf{x})_i} = \sum_j \frac{\partial l}{\partial \mathbf{f}(\mathbf{x})_j} \frac{\partial \mathbf{f}(\mathbf{x})_j}{\partial \mathbf{z}^o(\mathbf{x})_i}$$

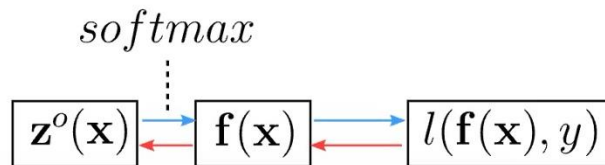
Chain rule!



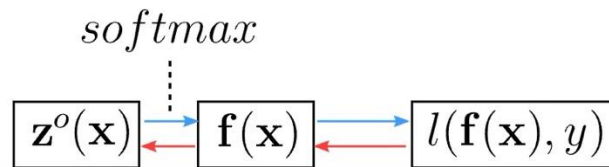
$$\begin{aligned}\frac{\partial l}{\partial \mathbf{z}^o(\mathbf{x})_i} &= \sum_j \frac{\partial l}{\partial \mathbf{f}(\mathbf{x})_j} \frac{\partial \mathbf{f}(\mathbf{x})_j}{\partial \mathbf{z}^o(\mathbf{x})_i} \\ &= \sum_j \frac{-1_{y=j}}{\mathbf{f}(\mathbf{x})_y} \frac{\partial \text{softmax}(\mathbf{z}^o(\mathbf{x}))_j}{\partial \mathbf{z}^o(\mathbf{x})_i}\end{aligned}$$

$$\frac{\partial l}{\partial \mathbf{f}(\mathbf{x})_i} = \frac{-1_{y=i}}{\mathbf{f}(\mathbf{x})_y}$$

$$\mathbf{f}(\mathbf{x}) = \text{softmax}(\mathbf{z}^o(\mathbf{x}))$$



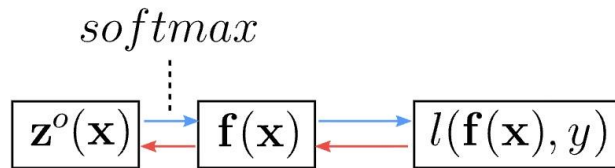
$$\begin{aligned}
 \frac{\partial l}{\partial \mathbf{z}^o(\mathbf{x})_i} &= \sum_j \frac{\partial l}{\partial \mathbf{f}(\mathbf{x})_j} \frac{\partial \mathbf{f}(\mathbf{x})_j}{\partial \mathbf{z}^o(\mathbf{x})_i} \\
 &= \sum_j \frac{-1_{y=j}}{\mathbf{f}(\mathbf{x})_y} \frac{\partial \text{softmax}(\mathbf{z}^o(\mathbf{x}))_j}{\partial \mathbf{z}^o(\mathbf{x})_i} \\
 &= -\frac{1}{\mathbf{f}(\mathbf{x})_y} \frac{\partial \text{softmax}(\mathbf{z}^o(\mathbf{x}))_y}{\partial \mathbf{z}^o(\mathbf{x})_i}
 \end{aligned}$$



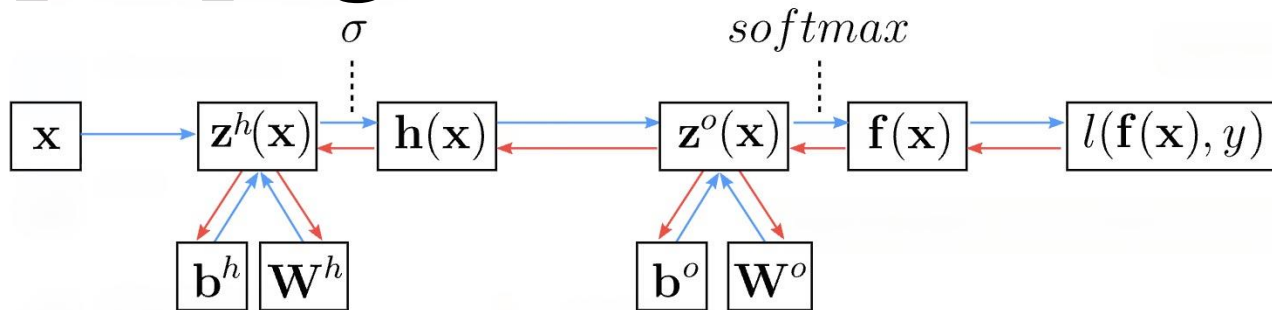
$$\begin{aligned}
\frac{\partial l}{\partial \mathbf{z}^o(\mathbf{x})_i} &= \sum_j \frac{\partial l}{\partial \mathbf{f}(\mathbf{x})_j} \frac{\partial \mathbf{f}(\mathbf{x})_j}{\partial \mathbf{z}^o(\mathbf{x})_i} \\
&= \sum_j \frac{-1_{y=j}}{\mathbf{f}(\mathbf{x})_y} \frac{\partial \text{softmax}(\mathbf{z}^o(\mathbf{x}))_j}{\partial \mathbf{z}^o(\mathbf{x})_i} \\
&= -\frac{1}{\mathbf{f}(\mathbf{x})_y} \frac{\partial \text{softmax}(\mathbf{z}^o(\mathbf{x}))_y}{\partial \mathbf{z}^o(\mathbf{x})_i} \\
&= \begin{cases} -\frac{1}{\mathbf{f}(\mathbf{x})_y} \text{softmax}(\mathbf{z}^o(\mathbf{x}))_y (1 - \text{softmax}(\mathbf{z}^o(\mathbf{x}))_y) & \text{if } i = y \\ \frac{1}{\mathbf{f}(\mathbf{x})_y} \text{softmax}(\mathbf{z}^o(\mathbf{x}))_y \text{softmax}(\mathbf{z}^o(\mathbf{x}))_i & \text{if } i \neq y \end{cases} \\
&= \begin{cases} -1 + \mathbf{f}(\mathbf{x})_y & \text{if } i = y \\ \mathbf{f}(\mathbf{x})_i & \text{if } i \neq y \end{cases}
\end{aligned}$$

$$\nabla_{\mathbf{z}^o(\mathbf{x})} l(\mathbf{f}(\mathbf{x}), y) = \mathbf{f}(\mathbf{x}) - \mathbf{e}(y)$$

$\mathbf{e}(y)$: one-hot encoding of y



Backpropagation

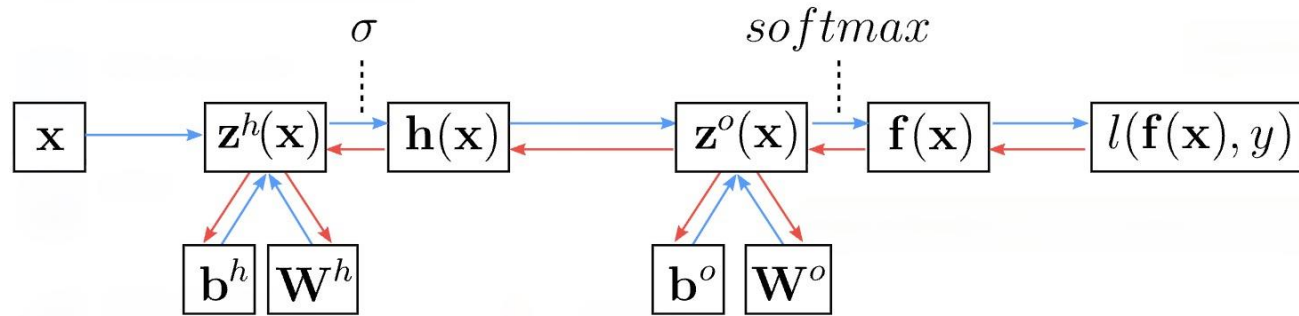


Gradients

- $\nabla_{\mathbf{z}^o(\mathbf{x})} l = \mathbf{f}(\mathbf{x}) - \mathbf{e}(y)$
- $\nabla_{\mathbf{b}^o} l = \mathbf{f}(\mathbf{x}) - \mathbf{e}(y)$

because $\mathbf{z}^o(\mathbf{x}) = \mathbf{W}^o \mathbf{h}(\mathbf{x}) + \mathbf{b}^o$ and then $\frac{\partial \mathbf{z}^o(\mathbf{x})_i}{\partial \mathbf{b}^o_j} = 1_{i=j}$

Backpropagation



Partial derivatives related to $\mathbf{W}^o$

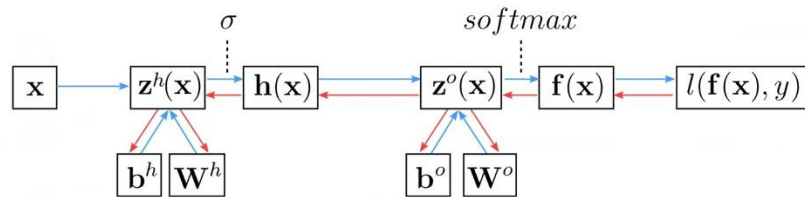
$$\bullet \frac{\partial l}{\partial \mathbf{W}_{ij}^o} = \sum_k \frac{\partial l}{\partial \mathbf{z}^o(\mathbf{x})_k} \frac{\partial \mathbf{z}^o(\mathbf{x})_k}{\partial \mathbf{W}_{ij}^o}$$

$$\bullet \nabla_{\mathbf{W}^o} l = (\mathbf{f}(\mathbf{x}) - \mathbf{e}(y)) \cdot \mathbf{h}(\mathbf{x})^\top$$

Backprop gradients

Compute activation gradients

- $\nabla_{\mathbf{z}^o(\mathbf{x})} l = \mathbf{f}(\mathbf{x}) - \mathbf{e}(y)$



Compute layer params gradients

- $\nabla_{\mathbf{W}^o} l = \nabla_{\mathbf{z}^o(\mathbf{x})} l \cdot \mathbf{h}(\mathbf{x})^\top$
- $\nabla_{\mathbf{b}^o} l = \nabla_{\mathbf{z}^o(\mathbf{x})} l$

Compute prev layer activation gradients

- $\nabla_{\mathbf{h}(\mathbf{x})} l = \mathbf{W}^{o\top} \nabla_{\mathbf{z}^o(\mathbf{x})} l$
- $\nabla_{\mathbf{z}^h(\mathbf{x})} l = \nabla_{\mathbf{h}(\mathbf{x})} l \odot \sigma'(\mathbf{z}^h(\mathbf{x}))$

Practice

- $x \in \mathbb{R}^2$, one hidden layer ($m = 2$), $N = 3$ classes
- Activation $\sigma = \text{ReLU}$; output uses **softmax**
- Cross-entropy loss $\ell(f(x), y) = -\sum_{i=1}^N y_i \log f_i(x)$

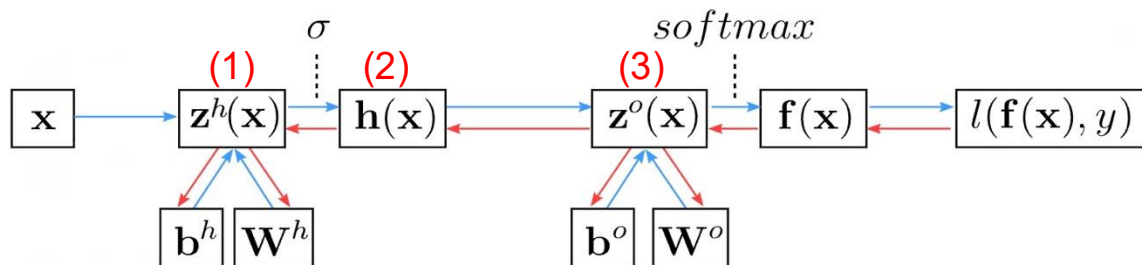
Parameters

$$W^h = \begin{bmatrix} 0.2 & -0.4 \\ 0.7 & 0.5 \end{bmatrix}, \quad b^h = \begin{bmatrix} 0.1 \\ -0.3 \end{bmatrix}$$

$$W^o = \begin{bmatrix} 0.6 & -0.1 \\ -0.2 & 0.9 \\ 0.1 & 0.2 \end{bmatrix}, \quad b^o = \begin{bmatrix} 0.05 \\ -0.05 \\ 0.02 \end{bmatrix}$$

$$x = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \quad y = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} \quad (\text{true class 1})$$

Practice - Forward Pass

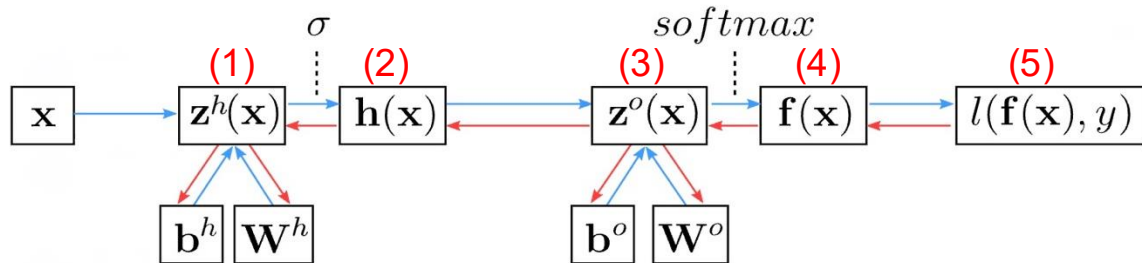


$$z^h = W^h x + b^h = \begin{bmatrix} -0.5 \\ 1.4 \end{bmatrix} \quad (1)$$

$$h = \sigma(z^h) = \max(0, z^h) = \begin{bmatrix} 0 \\ 1.4 \end{bmatrix} \quad (2)$$

$$z^o = W^o h + b^o = \begin{bmatrix} -0.09 \\ 1.21 \\ 0.30 \end{bmatrix} \quad (3)$$

Practice - Forward Pass



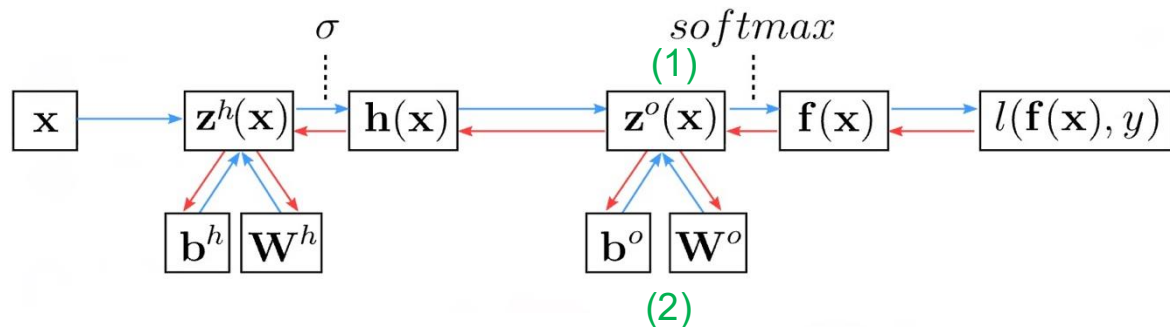
Softmax:

$$f = \text{softmax}(z^o) = \begin{bmatrix} 0.1627 \\ 0.5970 \\ 0.2403 \end{bmatrix} \quad (4)$$

Loss:

$$\ell = -\log f_1 \approx 1.8158 \quad (5)$$

Practice – Backward Pass



Output

$$\delta^o = \frac{\partial \ell}{\partial z^o} = f - y = \begin{bmatrix} -0.8373 \\ 0.5970 \\ 0.2403 \end{bmatrix} \quad (1)$$

$$\nabla_{z^o(\mathbf{x})} l(\mathbf{f}(\mathbf{x}), y) = \mathbf{f}(\mathbf{x}) - \mathbf{e}(y)$$

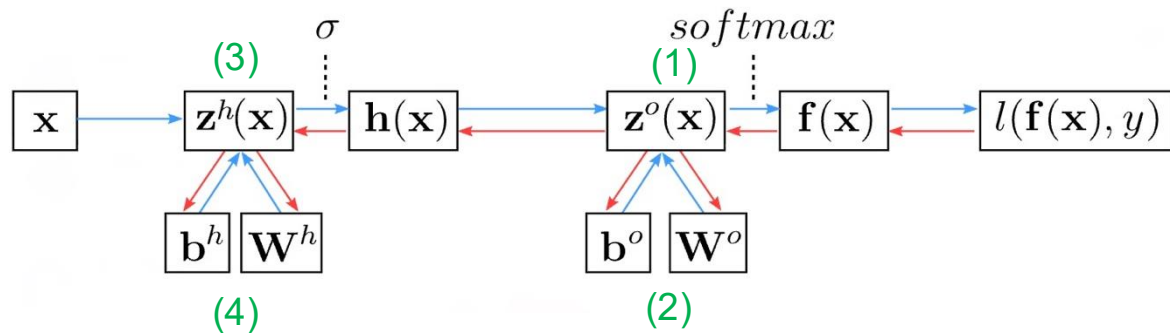
$\mathbf{e}(y)$: one-hot encoding of y

Output-layer gradients

$$\nabla_{W^o} \ell = \delta^o \mathbf{h}^\top = \begin{bmatrix} 0 & -1.1722 \\ 0 & 0.8358 \\ 0 & 0.3364 \end{bmatrix}, \quad \nabla_{b^o} \ell = \delta^o = \begin{bmatrix} -0.8373 \\ 0.5970 \\ 0.2403 \end{bmatrix} \quad (2)$$

- $\nabla_{\mathbf{W}^o} \ell = \nabla_{z^o(\mathbf{x})} \ell \cdot \mathbf{h}(\mathbf{x})^\top$
- $\nabla_{\mathbf{b}^o} \ell = \nabla_{z^o(\mathbf{x})} \ell$

Practice – Backward Pass



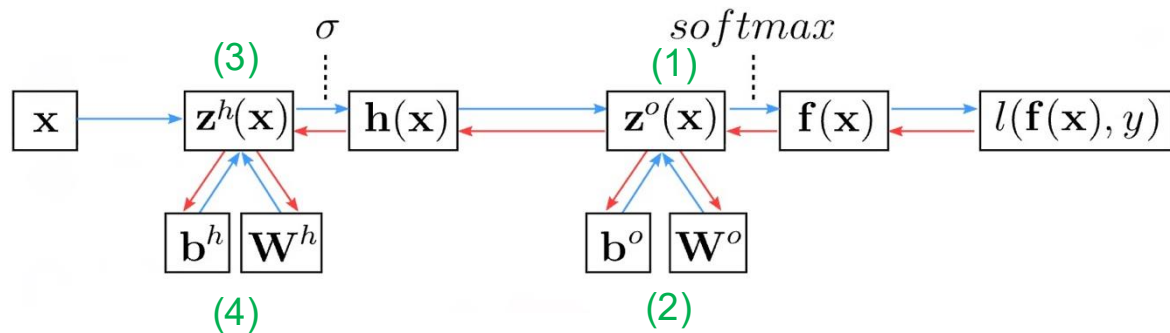
Backprop to hidden

$$\delta^h = \left((W^o)^\top \delta^o \right) \odot \sigma'(z^h) = \begin{bmatrix} 0 \\ 0.6691 \end{bmatrix} \quad (\text{ReLU mask } [0, 1]) \quad (3)$$

Hidden-layer gradients

$$\nabla_{W^h} \ell = \delta^h x^\top = \begin{bmatrix} 0 & 0 \\ 0.6691 & 1.3382 \end{bmatrix}, \quad \nabla_{b^h} \ell = \delta^h = \begin{bmatrix} 0 \\ 0.6691 \end{bmatrix} \quad (4)$$

Practice – Summary



- $\delta^o = \mathbf{f} - y$ (1)

- $\nabla_{\mathbf{W}^o} = \delta^o \mathbf{h}^\top$, $\nabla_{\mathbf{b}^o} = \delta^o$ (2)

- $\delta^h = (\mathbf{W}^o)^\top \delta^o \odot \sigma'(\mathbf{z}^h)$ (3)

- $\nabla_{\mathbf{W}^h} = \delta^h \mathbf{x}^\top$, $\nabla_{\mathbf{b}^h} = \delta^h$ (4)

Stochastic Gradient Descent (SGD)

In stochastic (or "on-line") gradient descent, the true gradient of $Q(w)$ is approximated by a gradient at a single sample:

$$w := w - \eta \nabla Q_i(w).$$

Learning Rate



WIKIPEDIA
The Free Encyclopedia

How to update weights during training?

Learning Rate

- Very sensitive:
 - Too high $\rightarrow$ early plateau or even divergence
 - Too low $\rightarrow$ slow convergence
 - Try a large value first: $\eta = 0.1$ or even $\eta = 1$
 - Divide by 10 and retry in case of divergence

Initialization

- Initializing W^h and W^o :

Why not all zeros?

Initialization

- Initializing W^h and W^o :
 - Zero is a saddle point: no gradient, no learning
 - Constant init: hidden units collapse by symmetry
 - Solution: random init, ex: $w \sim \mathcal{N}(0, 0.01)$
 - Better inits: Xavier Glorot and Kaming He & orthogonal
- Biases can (should) be initialized to zero

One Iteration (with SGD update)

1. Forward pass

- Compute hidden pre-activation

$$z^h = W^h x + b^h$$

- Apply activation ($\sigma = \text{ReLU}$):

$$h = \sigma(z^h)$$

- Compute output logits

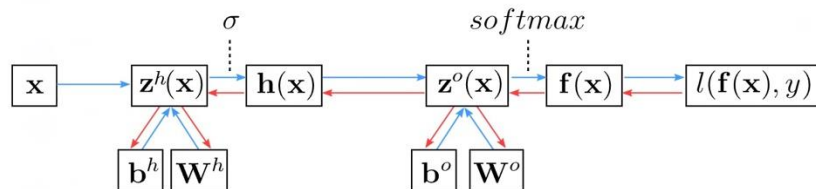
$$z^o = W^o h + b^o$$

- Softmax:

$$f = \text{softmax}(z^o)$$

- Compute loss (cross-entropy):

$$\ell = - \sum_{i=1}^N y_i \log f_i$$



One Iteration (with SGD update)

2. Backward pass

- Output

$$\delta^o = f - y$$

- Output layer gradients:

$$\nabla_{W^o} = \delta^o h^\top, \quad \nabla_{b^o} = \delta^o$$

- Hidden

$$\delta^h = (W^o)^\top \delta^o \odot \sigma'(z^h)$$

- Hidden layer gradients:

$$\nabla_{W^h} = \delta^h x^\top, \quad \nabla_{b^h} = \delta^h$$

One Iteration (with SGD update)

3. Update step (SGD with learning rate η)

$$\begin{aligned} W^o &\leftarrow W^o - \eta \nabla_{W^o}, & b^o &\leftarrow b^o - \eta \nabla_{b^o} \\ W^h &\leftarrow W^h - \eta \nabla_{W^h}, & b^h &\leftarrow b^h - \eta \nabla_{b^h} \end{aligned}$$

Next week

Workshop: Implement Backpropagation via PyTorch

Lectures: Regularisers in training